SYSTEMBESCHREIBUNG & ARCHITEKTURDOKUMENTATION
BIRDS AI SOUND CLASSIFIER

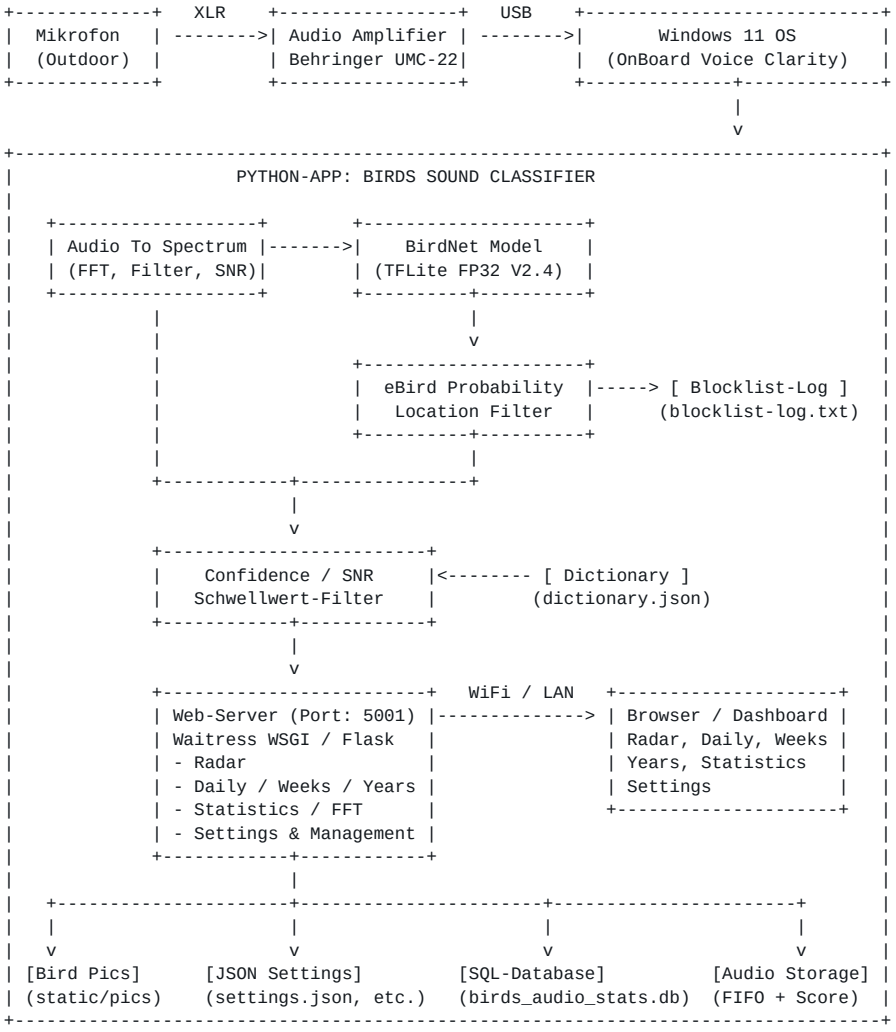
Projekt-Repository: https://github.com/mobifu1/birds_ai_sound_classifier
Entwicklungsansatz: Gemini Agent-based Vibecoding (Google DeepMind / Antigravity)
Betriebssystem: Microsoft Windows 11 OS
Stand / Version: V1.4.1-RC2 (November 2026)

1. EXECUTIVE SUMMARY & SYSTEMZWECK

Der "Birds AI Sound Classifier" ist ein hochgradig spezialisiertes, autonomes Bioakustik-System zur kontinuierlichen Echtzeit-Erfassung, Identifikation, Filterung, Archivierung und statistischen Auswertung von Vogelstimmen.

Das System kombiniert moderne physikalische Signalerfassung (XLR-Mikrofonie, Vorverstärkung), digitale Vorfilterung und Fast-Fourier-Transformation (FFT) mit einem Convolutional Neural Network (CNN) auf Basis des BirdNET-Modells unter TensorFlow / TFLite. Über ein zweistufiges Filtersystem (akustische Konfidenz + geografischer eBird-Wahrscheinlichkeitsabgleich) und fortgeschrittene Schwellwert-Logiken (SNR, Occurrence Threshold, Auto Season Lowering) liefert die Anwendung präzise ornithologische und bioakustische Echtzeitdaten. Die Ergebnisse werden über einen integrierten Webserver (Port 5001) in einem interaktiven Dashboard (akustisches 2D-Radar, Tages-, Wochen-, Jahresansichten, FFT-Periodizitätsanalysen, Territorien- und Nachtaktivitäts-Kacheln) im lokalen Netzwerk visualisiert.

2. SYSTEMARCHITEKTUR & SIGNALFLUSS ("HOW IT WORKS")



3. HARDWARE & SIGNALKETTE (BIOAKUSTIK-FRONTEND)

3.1 Mikrofon (Mic):

- Spezielles Außenmikrofon / Richtmikrofon zur Erfassung feiner bioakustischer Frequenzen im Bereich von ca. 200 Hz bis über 15.000 Hz.
- Symmetrische Signalführung über hochwertige XLR-Verkabelung zur Eliminierung von elektromagnetischen Einstreuungen über längere Leitungswege.

3.2 Audio-Vorverstärker & Interface (Amplifier):

- Behringer U-Phoria UMC-22 Audio Amplifier & Interface.
- Ausgestattet mit einem rauscharmen MIDAS-Design Mikrofonvorverstärker inklusive +48V Phantomspeisung für Kondensatormikrofone.
- Präzise Analog-Digital-Wandlung (ADC) und Weiterleitung per USB-Audio-Stream.

3.3 Betriebssystem (OS: Microsoft Windows 11):

- Robuste 64-Bit Host-Plattform für 24/7-Dauerbetrieb.
- OnBoard Voice Clarity / Windows Audio Stream: Native Abtastung und Vorfilterung über PyAudio / WASAPI.
- Prozess-Überwachung und Auto-Recovery über Windows-Subsysteme.

4. DIGITALE SIGNALVERARBEITUNG, FFT & AUDIO-VORFILTERUNG

4.1 Audio-Pipeline-Parameter:

- Sampling-Rate: 48.000 Hz (BirdNET Standard-Abtastrate)
- Kanal: 1 Kanal (Mono), 16-Bit PCM
- Blockgröße (Chunk): 1024 Samples
- Analyse-Fenster: 3,0 Sekunden (144.000 Samples)
- Überlappung (Overlapping): 1,5 Sekunden Versatz für lückenlose Überwachung.

4.2 Vorfilterung:

- High-Pass Filter: Herausfiltern von tiefen Störgeräuschen (z. B. Wind, Rumpeln, Nahverkehr, Infraschall) mit einstellbarer Grenzfrequenz (Standard: 500 Hz).
- Low-Pass Filter: Abschneiden hochfrequenter Artefakte (z. B. Grillen, Zirpen, Elektronikrauschen) mit konfigurierbarer Grenzfrequenz (Standard: 7.000 Hz).
- Rauschunterdrückung (Noise Reduction): Zuschaltbare dynamische Rauschminderung via 'noisereduce'-Bibliothek (Profile: Low, Medium, High).
- Signal-to-Noise Ratio (SNR): Kontinuierliche Berechnung des Signal-Rausch-Abstands in dB zur Bewertung der akustischen Signalqualität.

4.3 Spektrale Visualisierung & FFT-Analysen:

- Audio To Spectrum: Schnelle Fourier-Transformation (FFT) zur Erzeugung von Echtzeit-Wasserfall-Diagrammen (Waterfall Display im Dashboard).
- Forensische Phasenstabilität: Visualisierung der Phasendifferenzen (-180 Grad bis +180 Grad). Ermöglicht die Differenzierung zwischen organischen Rufen (wabernde, dynamische Phasenstrukturen) und technischen Störsignalen (starre, homogene Phasenlinien).
- Biologische FFT-Periodizitätsanalyse: Langzeit-FFT zur Erkennung biologischer Rhythmen der Vogelaktivität:
* 30-Tage-Periode: Lunarer Zyklus (Vogelzug, Nachtaktivität bei Vollmond).
* 24-Stunden-Periode: Zirkadianer Tag-/Nacht-Rhythmus.
* 12-Stunden-Periode: Morgen- und Abendchor (akustische Ausbreitungsoptima).
* 8h / 6h / 5h-Perioden: Stoffwechselzyklen, Siesta-Pausen, Fütterungsflüge und Brutwechsel-Intervalle.

5. KÜNSTLICHE INTELLIGENZ & BIRDNET-MODELL (TENSORFLOW / TFLITE)

5.1 KI-Modell & Runtime:

- BirdNET_GLOBAL_6K_V2.4_Model_FP32.tflite (Full-Precision 32-Bit Modell).
- Trainiert auf über 6.522 weltweite Vogelarten (Klassenliste: BirdNET Labels).
- Ausführung über TensorFlow / TFLite-Runtime, optimiert für latenzarme Inferenz auf CPU- oder GPU-Architekturen.

5.2 Zweistufiges Filterkonzept:

- Stufe 1: Akustische Konfidenz (Raw Confidence)
Das neuronale Netz analysiert das 3-Sekunden-Spektrogramm und gibt für jede Art einen Konfidenzwert (0 % bis 100 %) aus.
- Stufe 2: Geografischer Wahrscheinlichkeitsfilter (eBird Location Filter)
Abgleich der Kandidatenliste mit der globalen eBird-Beobachtungsdatenbank auf Basis der konfigurierten GPS-Koordinaten (Breite/Länge) und der Kalenderwoche (KW). Verhindert Verwechslungen heimischer Vögel mit exotischen Doppelgängern.

5.3 Schwellwerte & Plausibilisierung:

- Occurrence Threshold (OccT):
Historische eBird-Statistikhürde (Standard 0.03 = 3 % der Checklisten). Reduziert die 6.000 globalen Arten auf die ca. 50-150 saisonal realistischen Kandidaten am Standort.
- Auto Season Lowering (ASL):
Dynamische, kalenderwochenspezifische Absenkung des Schwellenwerts (gesteuert über 'auto_season_lowering.json'), um Zugvögel und späte Nachzügler im Herbst/Frühjahr nicht fälschlich abzuweisen.
- Force & Blocked Overrides (Wörterbuch):
* 'Force': Ignoriert den Geo-Filter für eine Art (z. B. für seltene Gäste).
* 'Blocked': Unterdrückt Arten gezielt (Ausschluss von Fehlalarmen).
- Blocklist-Logging ('blocklist-log.txt'):
Transparente Dokumentation aller Arten, die akustisch erkannt, aber durch den Geo-Filter oder Sperrlisten verworfen wurden (optional mit Pushover-Meldung für vom Modell blockierte Arten, die bereits im Dictionary hinterlegt sind).

6. INTELLIGENTE ERFASSUNGSLOGIK & SPAM-SCHUTZ

6.1 1-Fenster-Verzögerung mit Peak-Erkennung:

Erkennungen werden um 1,5 Sekunden (ein halbes Fenster) gepuffert. Das System vergleicht die Konfidenz des ersten Fensters mit dem direkt folgenden, überlappenden Fenster (in dem der Ruf oft zentriert liegt) und archiviert nur das Signal mit dem maximalen Konfidenzwert.

6.2 Intelligenter Streak-Spamschutz:

Ruft derselbe Vogel ununterbrochen über mehrere Fenster hinweg, wird er als "aktiver Streak" markiert und nicht mehrfach in die Datenbank geschrieben. Erst nach einer Pause / Stille wird der Streak zurückgesetzt.

6.3 Multitasking-Erkennung:

Rufen mehrere Vogelarten gleichzeitig im selben 3-Sekunden-Fenster, werden alle Arten parallel bewertet, auf Schwellwerte geprüft, getrennt gespeichert und auf dem Radar visualisiert.

7. WEBSERVER, DASHBOARD & BENUTZEROBERFLÄCHE (PORT 5001)

7.1 Backend-Architektur:

- Python Flask Webframework in Kombination mit Waitress Production WSGI Server.
- ZeroConf / mDNS-Broadcasting ('http://bird-ai-sound-classifier.local:5001').
- Multiprocessing-Architektur: Trennung von Audioaufnahme, KI-Inferenz und Webserver-Bedienung zur Vermeidung von Audio-Dropouts.

7.2 Dashboard-Module & Views:

- Akustisches 2D-Radar (-Radar):
Zweidimensionale Visualisierung aller aktuellen Erkennungen. Die Distanz zum Zentrum repräsentiert die Signalstärke (SNR), der Polarwinkel bzw. die Positionierung spiegelt die zeitliche Historie wider.
- Tagesansicht (-Daily):
Chronologische Timeline des aktuellen Tages mit Dämmerungs- und Sonnenstands-Berechnung (Astral / SunCalc.js).
- Wochen- & Jahresansichten (-Weeks / -Years / plausibility_check.html):
Aggregierte Balkendiagramme je Kalenderwoche, Gegenüberstellung mit eBird-Wahrscheinlichkeitskurven und ASL-Grenzen, CSV-Exportfunktion, interaktive "Zeitmaschine" (Ranking-Verlauf auf Gauß-Kurve mit gleitendem 4-Wochen-Sliding-Window) sowie unter "Erstmals erkannt" ein anklickbarer KI-Plausibilitätscheck (links für "Erstmals erkannt", rechts für "Zuletzt erkannt") auf einer eigenen Berichtsseite (plausibility_check.html) zur Bewertung von Edge-Cases (Tag/Nacht/Dämmerung, vorherrschendes GPS-Habitat wie Wald/Wasser/Moor, Signal-/Konfidenzbewertung sowie mögliche Begünstigung durch den Klimawandel und phänologische Verschiebungen).
- Spezies-Profile (-Statistics / species.html):
Detaillierte Einzelartenauswertungen inklusive 24h-Zeitradar, Frequenzspektrum, Phasenstabilität und Amplitudenverlauf.
- Territorien & Schnittmengen (-Territory / -Intersection):
Erfassung akustischer Revieraktivitäten und Co-Vorkommen überlappender Arten.
- Nachtaktivität & KI-Vorhersage (-Prediction / Kachel 7):
Spezialüberwachung nächtlicher Flug- und Zugrufe von Schwarm- und Zugvögeln sowie Google Gemini KI-gestützte Tages-, Seltenheits-, Zugvogel- und Nachtberichte.
- Einstellungen & Administration (-Settings):
Kindersicherung / Passwortschutz, Live-Audio-Pegelanzeige, Live-Spektrogramm, Audio-Filtereinstellungen, Wörterbuch-Duplettencheck, Zeitraum-basierte Falscherkennungs-Korrektur (mit verpflichtendem Vogelart-Filter), DB-Backups, Community-Versionsverteilung / Update-Statistik (CounterAPI.dev) sowie manuelles und vollautomatisches GitHub-Software-Update (Auto-Update täglich um 02:00 Uhr nachts, jeweils inkl. automatischem Vorab-DB-Backup).

8. PERSISTENZ, DATENHALTUNG & SCHNITTSTELLEN

8.1 SQL-Datenbank ('birds_audio_stats.db'):

- SQLite-Relationaldatenbank mit Indizierung auf Zeitstempel, Artname, SNR, Konfidenz und Dateipfade.
- Automatische Backup- und Konsistenzprüfungsroutinen (inkl. vollautomatischem Datenbank-Backup im Ordner 'backup_db/' vor jedem manuellen oder nächtlichen Software-Update).

8.2 Konfigurationsdateien (JSON):

- 'settings.json': Globale Laufzeitkonfiguration (Thresholds, GPS, Radar, Filter, Auto-Update, 'last_reported_version' für die Community-Versionsstatistik).
- 'dictionary.json': Vogelarten-Wörterbuch mit Übersetzungen, individuellem OccT, individueller Konfidenz, Phänologiestatus und Aufenthaltsmonaten.
- 'auto_season_lowering.json': Wochenspezifische Absenkungsparameter (geschützte Benutzerdatei bei Updates).
- 'pushover.json': Zugangsdaten und Filter für Smartphone-Push-Benachrichtigungen.
- 'birdweather.json': API-Token für das globale BirdWeather-Netzwerk.
- 'gemini.json': API-Token, Modellwahl, Wetteroption und Cache-Dauer für die Google Gemini KI-Integration ('prediction.html' und 'plausibility_check.html').

8.3 Duales Audio-Archiv (50 % FIFO-Ringpuffer & 50 % Highscore-Speicher):

- Begrenzung der Archivdateien je Vogelart zur Schonung von Speicherplatz.
- 50 % Chronologischer FIFO-Ringpuffer: Die neuesten Erkennungen bleiben erhalten.
- 50 % Exklusiver Highscore-Speicher: Speichert dauerhaft nur die akustischen Aufnahmen mit dem höchsten Score (Kombination aus Konfidenz und SNR). Bei DB-Löschungen oder Umbenennungen werden sowohl FIFO- als auch Highscore-Dateien synchron bereinigt.

8.4 Externe Schnittstellen & Benachrichtigungen:

- Pushover API: Push-Alerts aufs Smartphone (unterstützt Wildcards wie "-Specht", "Alle", "Neu" und individuelle Cooldown-Zeiten gegen Benachrichtigungsspam).
- BirdWeather API: Automatische Weiterleitung validierter Rufe an die weltweite Bioakustik-Forschungsplattform BirdWeather via Queue-Verzeichnis.
- Google Gemini API: KI-gestützte ornithologische Auswertungen, Nachtzug-Prognosen sowie Edge-Case-Plausibilitätschecks (Tageszeit/Dämmerung, GPS-Habitat, SNR/Konfidenz, Klimawandel-Einflüsse & phänologische Verschiebungen).
- CounterAPI v2 ('counterapi.dev'): Persistenter Online-Zählerdienst mit Bearer-Token-Authentifizierung ('COUNTERAPI_URL' / 'COUNTERAPI_TOKEN'). Meldet beim Start nach einem Versionswechsel ('last_reported_version != APP_VERSION') automatisch das Update ('/up') und stellt den aktuellen Zählerstand über '/api/community_versions' auf 'settings.html' im Bereich "Community Versionsverteilung" bereit.
- Autostart-Monitor ('autostart_monitor.pyw' / 'autostart_monitor.py', V1.2): Tkinter-basierter Watchdog (fensterloser Start via 'pythonw.exe') mit Prozessprüfung alle 60 Sekunden, automatischer Migration und Löschung veralteter 'autostart_monitor.exe'- und 'start.bat'-Dateien, Mehrfach-Instanzen-Bereinigung, Bereinigung verwaister Python- und CMD-Prozesse sowie direktem Anwendungsstart über 'python.exe' in einem eigenen, sichtbaren Konsolenfenster.

9. ENTWICKLUNGSPARADIGMA: GEMINI AGENT-BASED VIBECODING

Dieses Projekt entstand in einem modernen, agilen "Agent-based Vibecoding"-Prozess unter Einsatz von Google DeepMind Gemini-Modellen in Antigravity:

- Mensch-KI-Symbiose:
Der Entwickler steuerte Vision, biologisches Fachwissen, Zielarchitektur und praktische Tests mit echten Audioaufnahmen bei. Der Gemini-Coding-Agent übernahm die Codegenerierung, Signalverarbeitungs-Mathematik, Refactorings, CSS/JS-UI-Gestaltung und API-Integrationen in schnellen, iterativen Feedback-Schleifen.
- Cross-Machine Entwicklung:
Die Entwicklung erfolgte parallel auf zwei verschiedenen Arbeitsstationen (Entwicklungs-PC und Feld-/Bioakustik-Station). Während lokale Transkripte und Caches maschinengebunden sind, bildet das zentrale Git-Repository auf GitHub die vollständige, zusammenhängende Entwicklungsgeschichte ab.

10. PROJEKTSTATISTIKEN & METRIKEN

10.1 Entwicklungszeitraum & Meilensteine:

- Erster Git-Commit: 17. Juni 2026 (00:09:32 UTC)
- Aktueller Stand: 5. November 2026
- Gesamtdauer: 126 Tage (über 4 Monate kontinuierliche Weiterentwicklung)

10.2 Git-Repository & Commits:

- Repository: https://github.com/mobifu1/birds_ai_sound_classifier
- Gesamtanzahl Commits: 290+ Commits
- Monatliche Verteilung der Entwicklungszyklen:
* Juni 2026: 55 Commits (Initiales Klassifizierungssystem, PyAudio)
* Juli 2026: 53 Commits (Web-Dashboard, Radar, SNR-Kalkulation)
* August 2026: 88 Commits (Filterkaskaden, Wörterbuch, eBird-Geofence)
* September 2026: 63 Commits (ASL, Phasenanalyse, FFT-Periodizität)
* Okt./Nov. 2026: 31+ Commits (Nachtkacheln, Zeitmaschine, KI-Plausibilitätscheck, Auto-Update, CounterAPI)

10.3 Prompts & AI-Agent-Interaktionen:

- Auf dieser lokalen Maschine registriert:
* 18 Antigravity-Sessions
* 200+ User-Prompts (u. a. Einzelsessions mit bis zu 72 Prompts)
- Gesamtschätzung über beide Entwicklungsmaschinen (Cross-Machine Setup):
* Ca. 360 bis 460+ interaktive Vibecoding-Prompts über die gesamte Projektlaufzeit.
* Durchschnittlich ca. 1,3 bis 1,6 Prompts pro fertiggestelltem Git-Commit.

10.4 Umfang des Codes (Lines of Code - LoC):

```
- Python-Quellcode:              7.332 Zeilen
    * Hauptanwendung (birds_ai_sound_classifier.py): 6.522 Zeilen
    * Monitor-Watchdog (autostart_monitor.py / .pyw):  810 Zeilen (je 405 Zeilen)
- HTML-Templates (15 Web-Views): 6.645+ Zeilen
    * settings.html:       1.665 Zeilen
    * yearly.html:         1.474 Zeilen
    * index.html:            895 Zeilen
    * weekly.html:           480 Zeilen
    * species.html:          471 Zeilen
    * plausibility_check:    425 Zeilen
```

```
    * daily.html:            324 Zeilen
    * prediction.html:       296 Zeilen
    * intersection.html:     282 Zeilen
    * wiki.html:             270 Zeilen
    * delete_entry.html:     227 Zeilen
    * territory.html:        190 Zeilen
    * db_edit.html:          156 Zeilen
    * fft.html:              142 Zeilen
    * login/misc:             87 Zeilen
- Cascading Style Sheets (CSS):    420 Zeilen
- Client JavaScript (suncalc):     317 Zeilen
- NETTO-QUELLCODE (Codebase):   14.714+ ZEILEN
- BirdNET Klassen-Labels:        6.522 Zeilen (BirdNET 6K Labels)
- Konfigurations- & Textdateien:   910 Zeilen (Dictionary, Docs)
```

- GESAMTUMFANG PROJEKTDATEIEN: 22.146+ ZEILEN

Dokumentation generiert am: 5. November 2026

Autor & Maintainer: Andreas / mobifu1

Pair Programming: Gemini Agent via Google Antigravity